

Faster printf Debugging (Part 3)

Nathan Jones

Contents

- [Introduction](#)
- [A better metric](#)
- [Serialize your messages](#)
 - [MessagePack](#)
 - [FlatBuffers](#)
 - [bitproto](#)
- [Getting to ludicrous speed](#)
- [Conclusion](#)

Introduction

Debugging via `printf` (or `Serial.print` or, as I suggested in a previous article, `snprintf`) can be an extremely useful tool for seeing, quickly, what your system is doing and to zero in on the parts of your system that are, or could soon be, causing errors. This tool is not without its downsides, however, and you may have found in your experimentation with `printf` that calling it more than a few times **significantly slows down your program**. This challenge limits the times you can use `printf/snprintf` IRL, despite how useful it can be. In this article (as we did in [Parts 1 and 2](#)) we'll do our best to **make printf/snprintf blisteringly fast** so that you can use it to your heart's content!

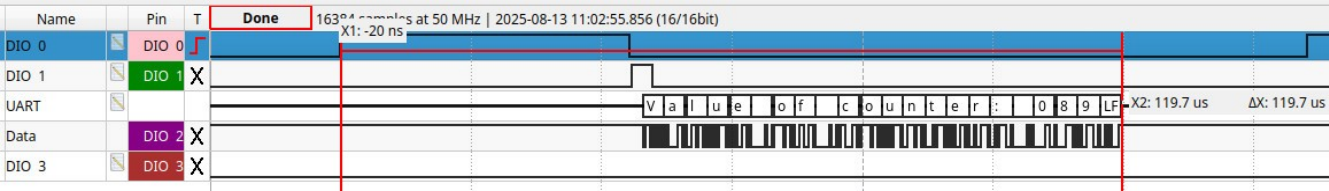
At the end of the last article, we used a buffered USB connection to improve the rate at which a Nucleo-F042K6 could send out a 22-character message (with one integer argument) from 7.14k messages per second (which we said had an execution time of 140 μ s) to 16.4k messages per second (an execution time of 88 μ s).

In this article, we'll look at a few more advanced optimization techniques that can further improve our message rate to **38.9k** or even **117.8k messages per second**!

A better metric

In the last two articles, we've been comparing the various ways to send out debug messages using their execution times. This made a ton of sense in [the first article](#) when we were still using `printf`, since it was a blocking function. The faster we could make that function execute, the more messages we would be able to send with whatever clock cycles were left over after our processor was done with its application tasks. At the end of the first article, we had reduced the execution time of `printf` (assuming a 22-character message with 1 integer argument) to just 140 μ s. At that rate, we could transmit **7.14k messages per second** (1 msg / 140 μ s = 7,142 msg/sec), minus any actual work that our processor was doing. (If our processor was only actually idle 40% of its time, for instance, then we would only be able to send out about 2.9k messages per second [40% of 7.14k].)

The notion of the “execution time” to send out a debug message became complicated, though, when we started sending out messages asynchronously, as we did in the [last article](#). I said then that the “execution time” using `snprintf`, for instance, was 120 μ s, as a result of calling `snprintf` (45 μ s) and subsequently sending out our message over UART at 3 Mbaud (75 μ s).

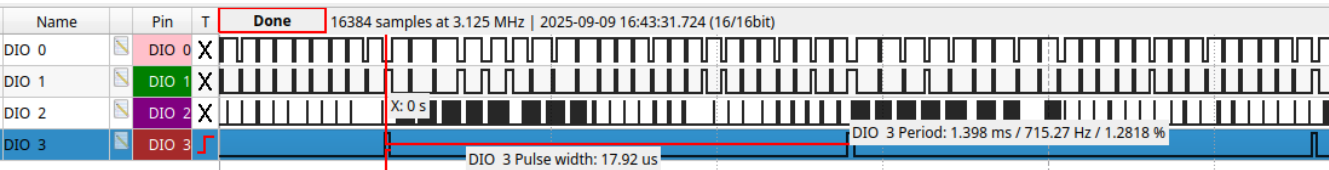


Although this was true in the sense that it took 120 μ s from start to finish to send out one of our debug messages, it *didn't* necessarily mean that we were limited to only sending out 8.3k messages per second (1 msg / 120 μ s = 8,333 msg/sec). This is because we are now sending messages *asynchronously*; in the 75 μ s it takes our first message to be sent out, our processor is already preparing another message. Or, rather, it *could* be preparing another message, only I didn't show it in my example. When I reworked my code to take advantage of this asynchrony, I discovered that it now took **74 μ s** in total to prepare a single message this way. (The increase in `snprintf`'s execution time from 45 to 68 μ s is the result of it being interrupted by the UART ISR 22 times, once per character in our message.)



What this means is that we can *actually* send out as many as **13.5k messages per second** (1 msg / 74 μ s = 13,513 msg/sec)!

So “messages per second” is a better metric for our purposes than “execution time”. Also, now that the two parts to sending out a message ([1] preparing it for transmission and [2] actually transmitting it) happen at the same time, we only really need to optimize the one that's taking longer. It was kind of a wash in the above example, so in that second article I decided (perhaps arbitrarily) to move to a faster means of communication: USB. After buffering our USB transmissions, I said in that article that our “execution time” was faster than before; down to 88 μ s! Again, this is not *quite* the way we want to look at things, since the “preparing of messages by the processor” overlapped with the “sending out of USB messages”, which was happening in hardware. It would have been better for me to have shown the *rate* at which messages were actually being sent out, which you can see below.



In this graphic, you can see DIO1 and DIO2 toggling once for each message added to the USB buffer. DIO3 toggles whenever we fill up one of those buffers and send it out over USB (and you can see the USB traffic afterward on DIO2). It's the *rate* of DIO3 that is most important to us, which shows us that we were able to send out 23 messages in 1.398 ms, a rate of **16.4k messages per second**. It takes just as long as before to prepare our messages (there's been no change to `snprintf`), so we wouldn't actually have expected this to have improved our message rate. However, it seems that the USB software, because it's so much faster than UART, is interrupting our processor less than the UART peripheral was and it's this that allows `snprintf` to complete more quickly than before, giving us a slightly higher message rate.

Looking at the screenshot above, it's clear that the *processor* is now the thing that's actually limiting how fast we can send out messages (you can see from the graphic above that it takes the processor almost twice as long to *put* 23 messages into the USB buffer as it does the USB peripheral to *send out* those 23 messages). We've already maxed out our processor's clock speed, so to help it prepare our messages faster we'll have to do something fairly creative!

Serialize your messages

Category	"Circumvent the Problem"			
Difficulty				
Rate increase	Varies (see below)			

In a previous article I suggested **shortening your debug messages** as a means of making `printf` "faster" (or, rather, of circumventing the problem of `printf`'s execution speed); a transfer rate of 100 kB/sec could represent only 100 messages that are 1 kB each but it could represent *1,000 messages* if those messages are only 100 B each. In fact, the logical conclusion of this technique is to **send a single byte corresponding to the message format being sent** (followed by any numeric values), effectively eliminating any calls to `printf/snprintf` entirely (though not entirely eliminating all of our formatting time). For example, we could assign the value `0x4D` to the string "[W] Core %dC" (which was used previously to indicate a fictional situation in which a "warp core" was experiencing an overtemperature condition) and then send out **0x4D 0x27 0x00** (0x0027 is 39 in hexadecimal). Doing this would transmit 10 fewer bytes than if we sent out the ASCII-version of the message, which would shorten the message by 77%, **quadrupling** our message rate in this fictional example! This series of bytes is distinctly *not* human-readable, of course, but it's possible to write a computer program to parse those values and reproduce the intended message after-the-fact. This technique is called "serializing", since we're serializing the binary values of any data that would be sent following each unique message ID. (Another term might be "tokenizing", since we're essentially converting each message into a "token" or "message ID" before they are transmitted.)

We can, potentially, send out many more messages this way but there are two problems that need to be solved before this technique is viable. The first has everything to do with the fact that we're now sending out *binary data* instead of ASCII characters. Consider our sample message above: `0x4D 0x27 0x00`. What if our temperature value was 77° C instead of 39° C? "77" is `0x4D` in hexadecimal,

so our message would become 0x4D 0x4D 0x00. But wait, how do we know that the second 0x4D isn't *actually* starting a new message? Or, put another way, what if our computer connects to our microcontroller right *after* the first 0x4D is sent and now it thinks that the second 0x4D really *is* starting a new message? We need a way to mark the beginning and/or end of our messages, but since our data values span the gamut of every possible binary value (0x00 to 0xFF), we need to be more creative than just putting a '\n' (0x0A) at the end of every message. The process of delimiting our messages is called **framing**.

One popular framing technique is **COBS** (consistent overhead byte stuffing). COBS uses 0x00 as the framing byte, replacing any actual 0x00s in your data stream with other numbers. For instance, the following string of values

0x01 0x04 **0x00** 0x08 0xD9 0x05 **0x00** 0xF4

is turned into *this* string of values once encoded using COBS

0x03 0x01 0x04 0x04 0x08 0xD9 0x05 0x02 0xF4 **0x00**

which you can test [here](#). Notice the absence of any 0x00s in the second string except at the very end; a receiving program can now look for any zero values to know where the start/end of each message is.

A big reason for the popularity of COBS is its low overhead: for any string of data with X bytes, COBS will only add $1 + \text{ceiling}(X/254)$ bytes to the final message.

The second problem is this: what does our computer *do* if it receives the COBS-encoded message 0x03 0x4D 0x27 0x01 0x00? You can follow the link above to decode that message into the following bytes: 0x4D 0x27 0x00. Ah, it's our overtemp message! Or, at least, it *looks* like it. What if we switch to a big-endian processor and now our message is 0x4D 0x00 0x27, though? What if we add a field to our message indicating whether we're reporting °F or °C and now our message is 0x4D 0x27 0x00 0x43 (0x43 is 'C' in ASCII)? We need a way for both systems to know the **format** of each message (down to how each individual byte is composed into the integers, floats, and strings that make up each message), so that every message *sent* is a message *received*.

A simple technique, like you can guess, is to just “hard-code” the data format into the program that's receiving our data. For our over-temp message above, we can essentially write code that says “If the first byte is 0x4D (overtemp message), the next two bytes are a `uint16_t` in little-endian order indicating the integer value of the temperature”. For small or bespoke systems, that solution will probably work fine (more on that later). But for larger projects, or for projects that need portability, this solution is brittle and error-prone.

A better technique is to use a known serialization format, such as [MessagePack](#) or [FlatBuffers](#). These standards define how a series of data values can be encoded into a string of bytes such that the receiving computer knows exactly how to reconstruct each message. They eliminate potential problems when sending and receiving from different processors (such as could happen if the processors differed in endianness or native bit-width) and they also have the benefit of being popular (and therefore having a lot of support and tooling).

MessagePack

Category	“Circumvent the Problem”				
Difficulty					
Rate increase	+19.9k messages/sec				

[MessagePack](#) is a serialization format that’s billed as being “like JSON, but smaller and faster.” It’s a schema-less format, like JSON, which means that every message contains all of the information needed to decode its contents.

What’s the difference between “schemaless” and “schema-based” serialization formats?

A “schema” is a description of how to interpret a series of bytes. For instance, what’s 0x64756E6B? If that’s supposed to represent a `uint32`, it might be the decimal value 1,685,417,579. If it’s a `float`, it could be 1.8109635e22. Heck, if it’s a series of ASCII values it could be the word “dunk”!

Some serialization formats say, “For this message, field 1 is a float.” Later on they might say to the receiver, “Hey! Field 1 is 0x64756E6B.” *Provided* the receiver knows that field 1 is a float, they can interpret this value correctly. This would be a “schema-based” serialization format, since the receiver needs to know ahead of time what the “schema” or format of each message is.

Schema files are usually written separately from the program and then a tool is used to create the proper program files (e.g. `.c`, `.cpp`, `.h`, `.py`, etc) based on that schema. Here’s a sample schema for the FlatBuffers format, which we’ll look at next:

```
namespace ThisExample;
```

```
table myMessage
{
    field1 : uint32;
    name : string;
}
```

```
root_type myMessage;
```

FlatBuffers uses the `flatc` tool to convert this schema into a corresponding `.c`, `.cpp`, or `.py` file (among many other languages). These files will have methods for creating and setting each field (`field1` and `name`) in `myMessage`.

Other serialization formats don’t define any schema ahead of time. Instead, they might just say, “Hey! The next four bytes should be read as a string: 0x64756E6B.” The receiver knows that this is exactly the word “dunk” and they *didn’t* need any prior information to deduce that. This is a “schemaless” format.

Using the [mpack](#) implementation of MessagePack for C, I created a simple array of two integers to send our sample message (“Value of counter: %03d”):

```
enum messageType_t { COUNTS, NUM_MSG_TYPES };
char buffer[MSG_LEN] = {0};
mpack_writer_t writer;
mpack_writer_init(&writer, buffer, MSG_LEN);
mpack_start_array(&writer, 2);
mpack_write_uint(&writer, COUNTS);
mpack_write_uint(&writer, counter);
mpack_finish_array(&writer);
```

where the first integer denotes this as the “Value of counter: %03d” message and the second integer is the value of `counter`. This is the equivalent of the JSON array `[0, 976]` (assuming `counter` is 976), and will be stored in memory as `0x92 0x00 0xCD 0x03 0xD0`, which you can test [here](#).

According to the [MessagePack specification](#), this translates to:

```
> 9: An array of N (<=15) elements follows
|> N = 2
||   > 0xxxxxxx: 1st element is a 7-bit positive integer (0)
||   |   > CD: 2nd element is a big-endian uint16 (0x3D0 = 976)
||   |   |
0x92 0x00 0xCD 0x03 0xD0
```

(A better use of the “schemaless” format would probably have been to have made a map (i.e. a dictionary) such as `{"id": COUNTS; "counter": 976}`, but this would, of course, have taken longer to pack and also more bytes to encode.)

This data was then COBS-encoded. COBS encoding was very straightforward using [this library](#):

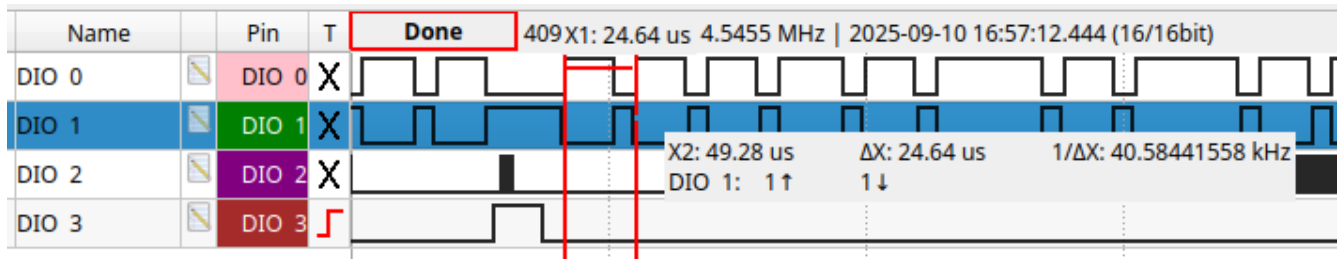
```
cobs_encode_result result = cobs_encode(dst, dst_len, src, src_len);
```

The COBS encoded form of our sample message above would then be `0x02 0x92 0x04 0xCD 0x03 0xD0 0x00`. (The library above won’t append the `0x00` delimiter, though, so don’t forget to add that yourself!)

The full encoding process, then, was to

1. Create a message using `mpack`
2. COBS-encode the `buffer` where the `mpack` message was stored
3. Send the resulting series of COBS-encoded bytes (with a `0x00` on the end!) out over USB.

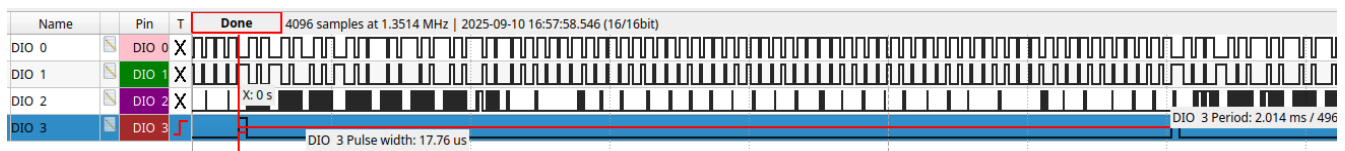
In my testing it took only 25 μ s to create each message in this way and add it to the USB buffer!



To decode this message, the receiver conducts the same process in reverse:

1. Read bytes until `0x00` is reached
2. Decode the COBS-encoded message
3. Use `mpack` to unpack the now decoded message into its various data fields

These messages were also very short: only **7 bytes long** (after they had been COBS encoded), meaning my 512 byte USB buffers could hold 73 of these messages. Sending these buffers out over USB happened every 2.01ms, which means that we achieved a message rate of 36.3k messages per second (73 messages / 2.01ms = 36,318 msg/sec) using `mpack`!



In fact, I was able to improve this slightly further to **38.9k messages per second** by removing the pin toggling code.

FlatBuffers

Category	“Circumvent the Problem”			
Difficulty				
Rate increase	About the same as <code>mpack</code>			

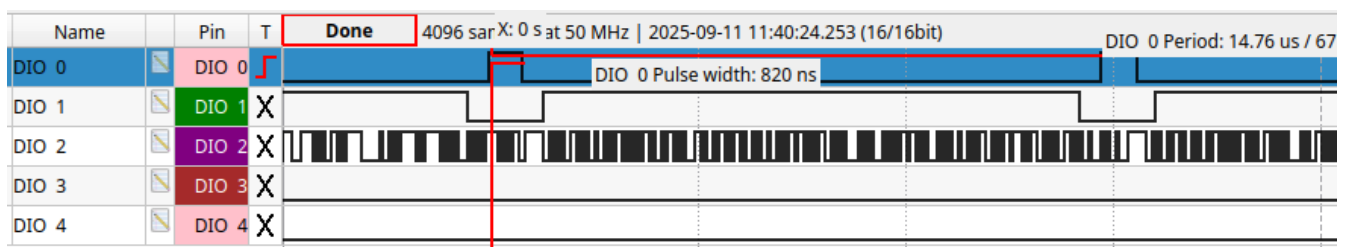
From the graphic above we can see that the length of time it takes to prepare a message using `mpack` is the current limiting factor preventing us from being able to send out more messages. At 25 μ s per message, the highest data rate we could expect with `mpack` is 40k messages per second, which is far less than the 142.9k messages per second we could hypothetically achieve if we could somehow prepare each message in the 7 μ s it would roughly take to send it out over USB. Needing a binary protocol that can encode/pack messages faster, we turn to [FlatBuffers](#). FlatBuffers is a schema-based binary protocol (so it uses a tool called `flatc` to turn your schemas into the appropriate program files; see “What’s the difference between ‘schemaless’ and ‘schema-based’ serialization formats?” above) with the unique feature that scalar values can be *modified in-place without needing to repack the message each time*, which we enable by passing the option `--gen-mutable` to the `flatc` compiler. This is huge! Using `mpack`, we had to make a new message each time the value of counter changed, which took 18 μ s (followed by 7 μ s to do the COBS encoding). With FlatBuffers, we can create a

COUNTS message once and be done; modifying the `counter` variable after that becomes as easy as modifying a normal struct member directly.

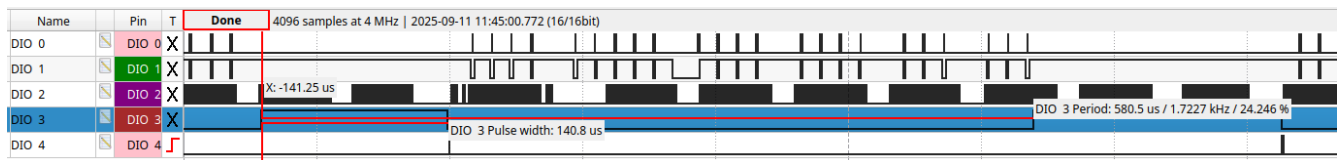
```
// Build a FlatBuffer message; buf is the uint8_t array with our data
auto counterMsg = messages::GetMutableCounter(buf);
```

```
// In a loop:
uint32_t next = counterMsg->value()+1;
counterMsg->mutate_value(next > 999 ? 0 : next);
```

In my tests, modifying `counterMsg->value` took only **820 ns** and required *no* subsequent repacking step! That's 22x faster than `mpack`!



Unfortunately, this didn't exactly yield the speed-up we wanted. To be sure, we're now pumping out messages as fast as our USB channel can handle them. Check out this screenshot, in which you can see the USB channel (DIO2) being used almost continuously.



Additionally you can see DIO3 being set for 140 μ s out of the 581 μ s it's taking to send out each USB buffer. DIO3 is set whenever our code is waiting for the USB to accept our next batch of data, so this means that, for the first time, we're making messages faster than we can send them out! Our processor utilization for this high data rate is only 75.9%, leaving the remaining 24.1% for our application tasks.

So what's the problem? Take a look at how many times DIO1 is toggled before we send out our USB buffer; this represents the number of messages we're able to fit in each 512 byte buffer. It's only toggling 23 times! Our FlatBuffer messages are *20 bytes long* (22 once they've been COBS-encoded), which means we only actually achieve a message rate of **39.6k messages per second** (23 messages / 581 μ s = 39,586 messages per second). Actually, it's slightly less than that, owing, I think, to occasional delays on the USB bus; in my tests, the average message rate over the course of one second was between **35 and 37k messages per second**, which may actually be slightly worse than before.

The silver lining, I suppose, is that we could theoretically achieve message rates as high as 60-70k messages per second if we switched to a communication channel that could handle 1.5 MBps or greater.

bitproto

Category	“Circumvent the Problem”			
Difficulty				
Rate increase	+78.9k messages/sec			

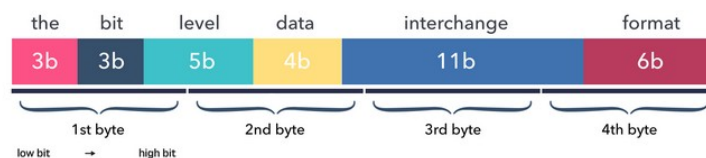
Okay, so we want a binary protocol that’s *fast* to pack and is *smaller* than 20 bytes (to store a single integer). At this point we’re even ready to make some questionable choices in the pursuit of speed! Enter [bitproto](#), a schema-based serialization format with the goals of being fast and allowing messages to store information down to the bit-level. Seriously; you can specify things like a “uint3” in your messages, which is a *3-bit unsigned integer*. The bitproto library handles bit-packing/-unpacking by shifting and masking your data values to fit into the fewest number of bytes necessary.

```
proto example

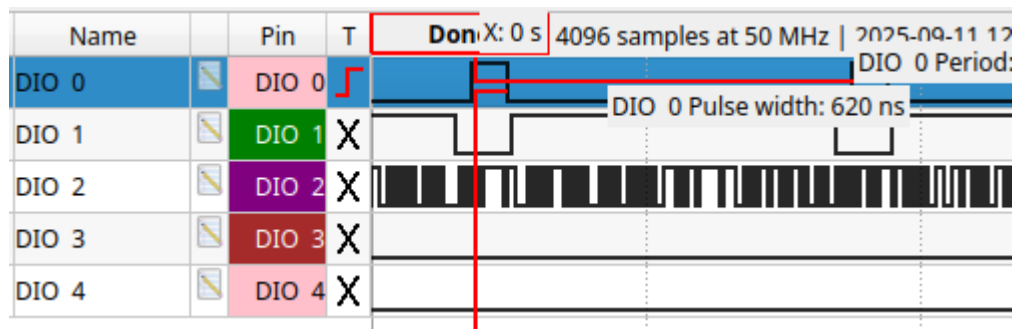
message Data {
  uint3 the = 1
  uint3 bit = 2
  uint5 level = 3
  uint4 data = 4
  uint11 interchange = 6
  uint6 format = 7
} // 32 bits => 4B
```

The **Data** above is called a message, it consists of 7 fields and will occupy a total of 4 bytes after encoding.

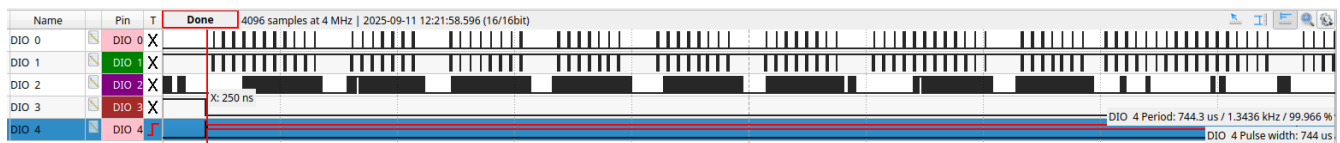
This image shows the layout of data fields in the encoded bytes buffer:



This makes for extremely small messages that are extremely fast to pack and unpack. Check out the screenshot below, which shows that it takes only **620 ns** to encode a bitproto message, 1.3x faster than for FlatBuffers!



Furthermore, since these messages have a single `uint32` in them (actually, I could have made them “`uint10`”s, since I reset `counter` to 0 as soon as it exceeds 999; that would have taken better advantage of all that bitproto has to offer!) they’re only 4 bytes long (6 bytes after COBS-encoding), so we can put 85 messages into each 512 byte USB buffer, achieving a message rate of 114.2k messages per second ($85 \text{ messages} / 744.3 \mu\text{s} = 114,201 \text{ messages/second}$)!



As with `mpack`, I was able to improve this slightly further to **117.8k messages per second** by removing the pin toggling code. Also, as with every other method so far except for `FlatBuffers`, our processor is fully utilized in order to achieve this data rate.

There are two downsides to keep in mind, however, that both result in `bitproto` maybe not being the best option for large projects or for projects that need to port to many different processor architectures.

The first is that `bitproto` assumes every processor has the same endianness; if the sender and receive differ (one is little-endian while the other is big-endian), you’ll get backwards data.

Second, `bitproto` stores no information inside each message about the contents therein. This means that *any* change to a message format (even if two fields are simply swapped around) would require recompiling the sending and receiving code so that they can each still access the fields correctly. By comparison, a `FlatBuffer` message can evolve to contain additional fields or for fields to move around within the message (following the rules [here](#)) and any code compiled against an older version will still work.

How would serializing work if I was also sending out the file, function, and line information for each debug message?

Great question! In a [previous article](#), I mentioned how useful it could be to include the file name, function name, and line number where a debug message occurred when it was printed.

```
[W] temp.c[56] adcRead() Core 39C
```

for instance, gives us much more information than

[W] Core 39C

especially if that same debug message occurs in multiple spots in our code. With a little bit of work, we can shorten the file/function/line information, too! The key is to replace all that with the value of the **Program Counter (PC)**, which also uniquely identifies every line of code in your program. You'd then use a program like [addr2line](#) on your host computer to convert each PC value to the appropriate file name, function name, and line number at a later time.

Using our last example, let's say the value of the PC was 0x8F4A081B when that debug message (" [W] Core 39C") was being printed. The actual values sent out over the UART/USB port (before COBS encoding) might be something like 0x8F 0x4A 0x08 0x1B 0x4D 0x00 0x27, representing first the PC (0x8F4A081B), then the message ID (0x4D), followed by the integer value for the message (0x0027).

Refer to your microcontroller's documentation to figure out how to access the PC. It may be accessible as a specific register, like on ARM, or you may need a more creative solution, such as writing a helper function that reads its return address and returns that value to the calling code. I hope you know how to read and write assembly!

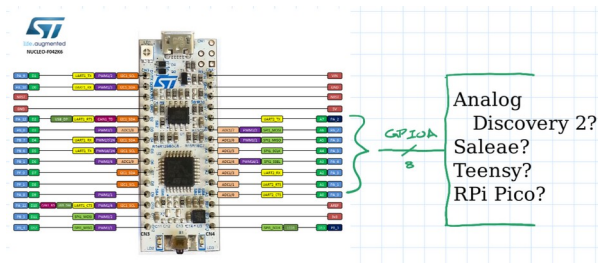
Getting to ludicrous speed



Can we get even faster?! Of course we can! Gigabit internet connections exist, right? Though, as you can imagine, doing so may require highly specialized hardware, like that found in your expensive wireless router.

Our last examples were pushing the limits of what our processor (STM32F042K6 at 48 MHz) and our connection (USB FS) could do. How could we make either or both of those faster? One unique idea is to **replace our USB connection with a GPIO port**. Instead of adding the bytes in our message to a buffer to be sent out over USB we would place each byte, in parallel, on one of our GPIO ports.

```
size_t parallel_write(const char * msg)
{
    char * start = msg;
    char c;
```



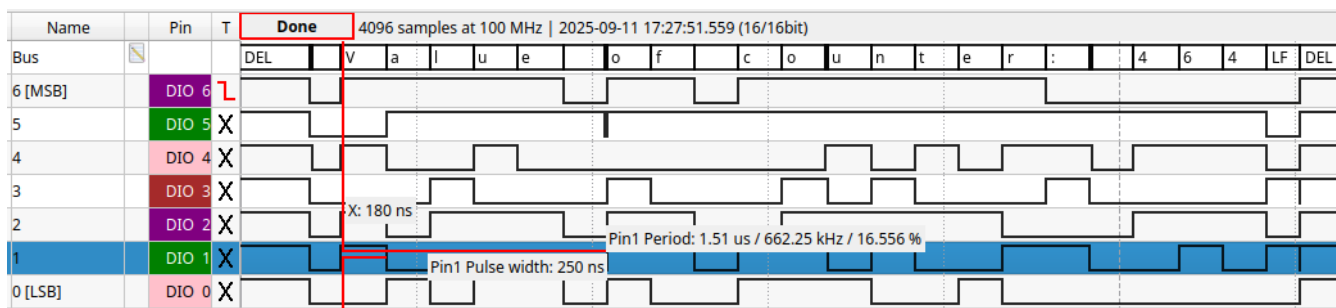
```
// GPIO lines idle high; set all to LOW to signal
// start of data transmission (just like a UART
// signal)
//
LL_GPIO_WriteOutputPort(GPIOA, (uint8_t)0x00);

// Place each byte of our message onto the GPIO
// port until NUL is reached.
//
while((c = *msg++) != '\0')
{
    LL_GPIO_WriteOutputPort(GPIOA, (uint8_t)c);
}

// Return GPIO lines to idle value (HIGH)
//
LL_GPIO_WriteOutputPort(GPIOA, (uint8_t)0xFF);

return (size_t)(msg-start-1);
}
```

The data rate of this method would only be limited by how fast you could toggle a GPIO pin, which, when I tested it, was as fast as **250 ns** on our current hardware. This would equate to a data rate of **32 Mbps**, roughly 2.6-4x faster than USB FS.



Of course, reading that data (especially if the messages were arriving at a rate of 117 KHz or higher) and then logging or printing it is another story! Your best bet is probably to capture that data using a high-speed logic analyzer such as an Analog Discovery 2 or Saleae. Many logic analyzers will allow you to record data to a file, which you could parse later on your computer. A Teensy or a Raspberry Pi Pico (using the PIO) *may* also be fast enough to capture that parallel data.

Other forms of high-speed communication include **SWO** (on Cortex-M3 and above), **USB 3.0**, and **Ethernet**, among many others. Utilizing one of those methods will require moving to a **more capable microcontroller** (sorry, STM32F0!), but that may also help you write out those messages even faster.

Lastly, I'd say we also haven't yet truly plumbed the optimization depths of our current examples! I mentioned in discussing bitproto that I could have stored our counter as a uint10 instead of a uint32. That would have required two fewer bytes to encode, possibly increasing our message rate to 164.9k messages per second ($117.8 * 7/5 = 164.9k$). Maybe [manual escaping and framing](#) would take less time than COBS encoding. Are we sure that the mpack/FlatBuffers/bitproto libraries couldn't be optimized to execute more quickly? I'm not sure that we are. :)

Conclusion

Although it took a lot of work, we were ultimately able to make significant improvements in how fast we could send messages from our microcontroller! By removing the debug string entirely, replaced with a single byte or “token” to represent which message was being sent, we could *significantly* shorten not just our processing time but also how long each message took to transmit. In the end, our “Value of counter: %03d” message could be reduced to just 7 bytes, representing (more or less)

- a single byte to indicate the message type,
- four bytes for the uint32 value of `counter`,
- two bytes to COBS encode the message, and
- one byte to terminate the message

(though the exact format varied depending on whether we used MessagePack, FlatBuffers, or bitproto). This allowed us to achieve message rates as high as **38.9k messages per second**, or even as high as **117.8k messages per second** provided we weren’t concerned with the portability or backwards compatibility of our messages. Getting to even higher message rates is completely possible but would require a faster processor, dedicated high-speed hardware peripherals, or both.

Here’s a summary of how far we’ve come.

Example	Difficulty	Function	Clock (MHz)	Baud rate	HAL / LL	Opt. Level	Rate ¹ (msg/sec)	Processor utilization
Baseline	N/A	printf	8	38400	HAL	-O0	142	100%
Faster clock		printf	48	38400	HAL	-O0	168	100%
Faster baud		printf	48	3000000	HAL	-O0	3,300	100%
Ditch the HAL		printf	48	3000000	LL	-O0	7,142	100%
Async		snprintf	48	3000000	LL	-O0	Unk	100%
-O3		snprintf	48	3000000	LL	-O3	13,513	100%
USB		snprintf	48	12000000	N/A	-O3	Unk	100%
Buffer It		snprintf	48	12000000	N/A	-O3	16,452	100%
mpack		See API	48	12000000	N/A	-O3	38,900	100%
FlatBuffers		See API	48	12000000	N/A	-O3	~36,000	75.9%
bitproto		See API	48	12000000	N/A	-O3	117,800	100%

1: For sending out a 22 character message (or equivalent token) with one integer

I hope that you’re able to incorporate one or more of the techniques above to get lots of debug messages off of your microcontroller in the future and that those debug messages give you much useful information.

If you’ve made it this far, thanks for reading and happy hacking!